Hntr

VAPT Report

Table of Contents

Document Detail

Document:	Web Application VAPT Report
Report Content:	Summarized Technical Report
Classification:	Confidential
Status:	Initial Report
Version:	1.0
Date:	30th Aug 2026
Approved By:	Mr. Aaryan Saharan

Report History

ISSUE DATE	VERSION	DESCRIPTION	PREPARAED BY	
30th Aug 2026	1.0	Initial Report	Mr. Aaryan Saharan	

Executive Summary:

Multiple API endpoints were found to be vulnerable to Broken Object Level Authorization (BOLA/IDOR). By manipulating user-controlled identifiers such as wallet addresses and usernames, an attacker could access sensitive information belonging to other users without authorization. The exposed data includes PII, financial records, transaction history, wallet balances, reward information, leadership payouts, network hierarchy, notifications, and internal business metrics. Collectively, these issues present a Critical risk due to the potential for large-scale unauthorized data harvesting and compromise of user confidentiality.

Scope:

The scope of the assessment included a comprehensive evaluation of web application and its APIs. These systems were selected due to their critical nature and the potential impact on the organization's overall security.

Methodology

A Grey-box testing approach was adopted for this assessment, involving thorough manual testing. The tools used included Burp Suite Pro, Nessus, and Acunetix to aid in identifying potential vulnerabilities.

Overall Review summary

Vulnerability Assessment and Penetration Tester of the Application revealed weaknesses. We observed that out of total **8** gaps; **8 Critical** Vulnerabilities were Found.

Refer to the tabular summary and pie chart of severity wise vulnerabilities identified during testing activity:

Severity/Risk	Count	% Of Total Count	Open
Critical	8	NA	8
High	0	NA	0
Medium	0	NA	0
Low	0	NA	0
Total Count	**8**	**NA**	**8**

Severity Metrics

Critical	Critical severity issues allow an attacker run arbitrary code on the underlying platform with the user's privileges in the normal course of usage. The defect that results in the termination of the complete system or one or more component of the system and causes extensive corruption of the data. The failed function is unusable and there is no acceptable alternative method to achieve the required results then the severity will be stated as critical.
High	These findings identify conditions that could directly result in the compromise or unauthorized access of a network, system, application or information. Examples of High Risks include known buffer overflows, weak or no passwords, no encryption, which could result in denial of service on critical systems or services; unauthorized access; and disclosure of information.
Medium	These findings identify conditions that do not immediately or directly result in the compromise or unauthorized access of a network, system, application or information, but do provide a capability or information that could, in combination with other capabilities or information, result in the compromise or unauthorized access of a network, system, application or information. Examples of Medium Risks include unprotected systems, files, and services that could result in denial of service on non-critical services or systems; and exposure of configuration information and knowledge of services or systems to further exploit.
Low	These findings identify conditions that do not immediately or directly result in the compromise of a network, system, application, or information, but do provide information that could be used in combination with other information to gain insight into how to compromise or gain unauthorized access to a network, system, application or information. Low risk findings may also demonstrate an incomplete approach to or application of security measures within the environment. Examples of Low Risks include cookies not marked secure; concurrent sessions and revealing system banners.

Detailed Findings

1. Insecure Direct Object Reference (IDOR) / Broken Object Level Authorization

Severity: Critical
CWE: CWE-639 – Authorization Bypass Through User-Controlled Key
OWASP API Security: API1:2023 – Broken Object Level Authorization
Affected Endpoint: `GET /api/users/wallet/{walletAddress}`

Description

The `/api/users/wallet/{walletAddress}` API endpoint is vulnerable to **Insecure Direct Object Reference (IDOR)** due to insufficient object-level authorization controls.

The endpoint accepts a wallet address as a user-controlled parameter and returns the corresponding user's profile information. During testing, the wallet address was modified from the originally requested user's wallet address to the wallet address of another user.

The application processed the modified request successfully and returned the second user's profile information with an HTTP `200 OK` response, without enforcing authorization to verify whether the requesting user was permitted to access the requested profile.

Steps to Reproduce

Step 1 — Request a user's profile using their wallet address

```
GET /api/users/wallet/0x4E6DcF5BbB225FFF0437A358d638bEAD891C07B8 HTTP/2
Host: hntr-backend-dev.onrender.com
```

The application returned the corresponding user's profile information.

Step 2 — Modify the wallet address

The wallet address was changed to another valid user's wallet address:

```
GET /api/users/wallet/0x883eae7dc8ae8ee5be23ab552ed36cc9a80c3a17 HTTP/2
Host: hntr-backend-dev.onrender.com
```

Step 3 — Observe the response

The server returned:

```
HTTP/2 200 OK
Content-Type: application/json
```

The response contained the profile of the other user, including sensitive information such as:

```
{
  "username": "user1",
  "walletAddress": "0x883eae7dc8ae8ee5be23ab552ed36cc9a80c3a17",
  "email": "user1@gmail.com",
  "phone": "+917264865086",
  "sponsorUsername": "adarsh",
```

```
  "ancestors": ["admin", "adarsh"],
  "directDownline": ["user2", "user3"],
  "tier": "Bronze",
  "rank": "None",
  "teamVolume": 21650,
  "legVolumes": {
    "user2": 21600,
    "user3": 50
  },
  "hntrPoints": 15775,
  "joinedAt": "2026-08-02T16:43:00.713Z"
}
```


This confirms that modifying the object identifier allows unauthorized access to another user's data.

Impact

Successful exploitation allows an attacker to access profiles belonging to other users by supplying their wallet addresses.

The exposed information includes **Personally Identifiable Information (PII)** such as:

- Email addresses
- Phone numbers
- Usernames
- Wallet addresses

Additionally, the API exposes sensitive business and account information, including:

- Sponsor relationships
- Ancestor hierarchy
- Direct downline information
- Membership tier and rank
- Team and leg volumes
- HNTR points
- Account creation information
- Feature entitlement status

An attacker capable of obtaining or enumerating wallet addresses could potentially **harvest user profiles at scale**, resulting in significant unauthorized disclosure of PII and user/account information.

Recommendation

Implement robust **server-side object-level authorization** for the affected endpoint.

The application should:

1. Authenticate the requesting user before processing the request.
2. Verify that the authenticated user has permission to access the requested wallet/profile.
3. Never treat possession or knowledge of a wallet address as authorization to access the associated profile.
4. Return `403 Forbidden` or an appropriate authorization response when unauthorized access is attempted.
5. Review all APIs that accept user-controlled identifiers such as wallet addresses, user IDs, usernames, or other object references for similar authorization weaknesses.
6. Apply the principle of **least privilege** and return only the information required for the intended functionality.

7. Implement appropriate monitoring and rate limiting to reduce the possibility of automated enumeration and bulk data harvesting.

Expected Result

A user should only be able to retrieve profile information for accounts they are explicitly authorized to access. Modifying the wallet address to another user's wallet address should **not** disclose that user's PII or other sensitive account information.

2. Insecure Direct Object Reference (IDOR) / Broken Object Level Authorization – Financial Information Exposure

Severity: Critical
CWE: CWE-639 – Authorization Bypass Through User-Controlled Key
OWASP API Security: API1:2023 – Broken Object Level Authorization
Affected Endpoint: `GET /api/network/{walletAddress}/points`

Description

The `/api/network/{walletAddress}/points` API endpoint is vulnerable to **Insecure Direct Object Reference (IDOR)** due to insufficient object-level authorization.

The endpoint accepts a wallet address as a user-controlled path parameter and returns the associated user's points summary and complete transaction ledger. During testing, the wallet address was modified from one user's wallet address to another user's wallet address.

The application processed the modified request successfully and returned the second user's financial and transaction-related information with an HTTP `200 OK` response, without verifying whether the requesting user was authorized to access the requested wallet's data.

Steps to Reproduce

Step 1 — Request the points information for a valid wallet

```
GET /api/network/0x883eae7dc8ae8ee5be23ab552ed36cc9a80c3a17/points HTTP/2
Host: hntr-backend-dev.onrender.com
```

The application returned the points summary and transaction ledger associated with the supplied wallet address.

Step 2 — Modify the wallet address

The wallet address was changed to another user's wallet address:

```
GET /api/network/0x4e6dcf5bbb225fff0437a358d638bead891c07b8/points HTTP/2
Host: hntr-backend-dev.onrender.com
```

Step 3 — Observe the response

The application returned:

```
HTTP/2 200 OK
Content-Type: application/json
```

The response disclosed the target user's points and transaction information, including:

```
{
  "hntrPoints": 645925,
  "ledger": [
    {
      "walletAddress": "0x4e6dcf5bbb225fff0437a358d638bead891c07b8",
      "amount": 250000,
      "source": "MEMBERSHIP_UPGRADE",
```

```
      "timestamp": "2026-08-27T15:22:28.124Z",
      "txHash": "0xde608a1096fa974f6a2c045a29f92cf44ced5a49ec04ba4e6bf366fd44339893",
      "usdValue": 1000
    }
  ]
}
```


The ledger contained numerous additional entries exposing commission earnings, membership purchases/upgrades, transaction hashes, transaction amounts, timestamps, and associated wallet information.

Impact

Successful exploitation allows an unauthorized user to retrieve another user's **financial and transaction-related information** by modifying the wallet address in the request.

The exposed information includes:

- Total HNTR points
- Commission earnings
- Commission levels
- Membership purchases
- Membership upgrades
- Transaction amounts
- USD-equivalent transaction values
- Transaction hashes
- Transaction timestamps
- Wallet addresses
- Transaction/ledger identifiers
- Financial activity associated with the account

This information could allow an attacker to profile users' financial activity, membership history, earnings, and transaction behavior.

Because the endpoint exposes **sensitive financial information belonging to another user without authorization**, and the object identifier is directly controllable through the request, the issue presents a **Critical** confidentiality impact. If wallet addresses can be obtained or enumerated, the vulnerability could potentially be exploited to harvest financial information across multiple accounts.

Recommendation

Implement strict **server-side object-level authorization** for the affected endpoint.

The application should:

1. Authenticate the requesting user before providing points or ledger information.
2. Verify that the authenticated user is authorized to access the requested wallet's financial information.
3. Do not consider knowledge of a wallet address sufficient authorization.
4. Return `403 Forbidden` or an equivalent authorization response when unauthorized access is attempted.
5. Review all endpoints accepting wallet addresses or other user-controlled identifiers for similar IDOR/BOLA vulnerabilities.
6. Restrict API responses to the minimum information required for the application's functionality.
7. Implement appropriate rate limiting and monitoring to detect automated wallet/profile enumeration and bulk data extraction.

Expected Result

Changing the wallet address to another user's wallet should **not** return that user's points, financial records, transaction history, or other account-specific information unless the authenticated requester has explicit authorization to access it.

3. Insecure Direct Object Reference (IDOR) / Broken Object Level Authorization – Unauthorized Transaction History Exposure

Severity: Critical
CWE: CWE-639 – Authorization Bypass Through User-Controlled Key
OWASP API Security: API1:2023 – Broken Object Level Authorization
Affected Endpoint: `GET /api/network/transactions/{walletAddress}`

Description

The `/api/network/transactions/{walletAddress}` API endpoint is vulnerable to **Insecure Direct Object Reference (IDOR)** due to insufficient object-level authorization.

The endpoint accepts a wallet address as a user-controlled path parameter and returns the transaction history associated with that wallet. During testing, the wallet address was modified from one user's wallet address to another user's wallet address.

The modified request was successfully processed and returned the target user's transaction history with an HTTP `200 OK` response, indicating that the application does not adequately verify whether the requesting user is authorized to access the requested wallet's transaction information.

Steps to Reproduce

Step 1 — Request transaction history for a valid wallet

```
GET /api/network/transactions/0x883eae7dc8ae8ee5be23ab552ed36cc9a80c3a17?limit=100 HTTP/2
Host: hntr-backend-dev.onrender.com
```

The application returned transaction records associated with the supplied wallet.

Step 2 — Modify the wallet address

The wallet address was changed to another user's wallet:

```
GET /api/network/transactions/0x4E6DcF5BbB225FFF0437A358d638bEAD891C07B8?limit=100 HTTP/2
Host: hntr-backend-dev.onrender.com
```

Step 3 — Observe the response

The modified request returned:

```
HTTP/2 200 OK
Content-Type: application/json
```

The application disclosed the target user's transaction history, including records such as:


```
{
  "type": "COMMISSION_CLAIM",
  "txHash": "0xad6a7dbb0ac483ed021f14894459ea40102242b1af1e2e3e2fb7e24dcb9c675f",
  "timestamp": "2026-08-10T16:49:46.829Z",
  "amount": "486.00",
  "token": "0xff26bf42e258979e307b581f32a7c984bceda66a",
  "status": "CONFIRMED"
```

```
}
```

The response further exposed transaction records associated with commissions, claims, membership upgrades, purchases, achievement bonuses, and leadership payouts.

Several `COMMISSION_EARNED` records additionally exposed information about other users participating in the commission structure, including their **wallet addresses and usernames**, along with commission amounts, locked amounts, transaction hashes, and hierarchy levels.

Impact

Successful exploitation allows an unauthorized user to access another user's **complete transaction history and financial activity** by modifying the wallet address.

The exposed information includes:

- Commission claim records
- Commission earnings
- Commission amounts and locked amounts
- Membership purchases and upgrades
- Achievement bonuses
- Leadership payouts
- Transaction hashes
- Transaction timestamps
- Token addresses
- Transaction status
- Membership tiers
- Commission hierarchy levels
- Source wallet addresses
- Source usernames

The endpoint therefore exposes not only the target user's financial activity but, through commission records, information relating to **other participants in the user's network**.

An attacker who can obtain or enumerate wallet addresses could potentially automate requests and **harvest transaction and financial information across multiple accounts**. This may result in significant confidentiality and privacy impact and could expose sensitive financial/business relationships within the platform.

Recommendation

Implement strict **server-side object-level authorization** for the affected endpoint.

The application should:

1. Authenticate the requesting user before returning transaction history.
2. Verify that the authenticated user is explicitly authorized to access the requested wallet's transaction records.
3. Do not treat knowledge of a wallet address as sufficient authorization.
4. Return `403 Forbidden` or an equivalent authorization response for unauthorized wallet access.
5. Review all endpoints accepting wallet addresses or other user-controlled identifiers for similar IDOR/BOLA vulnerabilities.
6. Apply the principle of least privilege and return only transaction information required by the authenticated user.

7. Avoid exposing unnecessary third-party information such as source usernames and wallet addresses in transaction records.
8. Implement rate limiting and monitoring to detect automated wallet enumeration and bulk transaction-history extraction.

Expected Result

Changing the wallet address to another user's wallet should **not** return that user's transaction history, financial activity, or associated network information unless the requesting user has explicit authorization to access it.

4. Insecure Direct Object Reference (IDOR) / Broken Object Level Authorization – Unauthorized Network Tree Exposure

Severity: Critical
CWE: CWE-639 – Authorization Bypass Through User-Controlled Key
OWASP API Security: API1:2023 – Broken Object Level Authorization
Affected Endpoint: `GET /api/network/{username}/tree`

Description

The `/api/network/{username}/tree` API endpoint is vulnerable to **Insecure Direct Object Reference (IDOR)** due to insufficient object-level authorization.

The endpoint accepts a username as a user-controlled path parameter and returns the user's network hierarchy. During testing, the username was modified from `user1` to `adarsh`.

The application successfully processed the modified request and returned the complete network tree associated with the other user, including information belonging to multiple members of that user's network.

No effective authorization control was observed to verify whether the requesting user was permitted to access the requested user's network hierarchy.

Steps to Reproduce

Step 1 — Request the network tree of a valid user

```
GET /api/network/user1/tree?depth=3 HTTP/2
Host: hntr-backend-dev.onrender.com
```

The application returned the network tree associated with `user1`.

The response included the user's wallet address, tier, rank, personal volume, and details of members within the network.

Step 2 — Modify the username

The username was changed to another user's username:

```
GET /api/network/adarsh/tree?depth=3 HTTP/2
Host: hntr-backend-dev.onrender.com
```

Step 3 — Observe the response

The application returned:

```
HTTP/2 200 OK
Content-Type: application/json
```

The response disclosed the target user's network hierarchy, including:

```
{
  "username": "adarsh",
```

```
  "walletAddress": "0x4e6dcf5bbb225fff0437a358d638bead891c07b8",
  "tier": "Platinum",
  "rank": "Hunter",
  "personalVolume": 1500,
  "children": [
    {
      "username": "user1",
      "walletAddress": "0x883eae7dc8ae8ee5be23ab552ed36cc9a80c3a17",
      "tier": "Bronze",
      "rank": "None",
      "personalVolume": 50,
      "children": [...]
    }
  ]
}
```


The returned hierarchy further disclosed multiple downstream users, including their **usernames, wallet addresses, membership tiers, ranks, and personal volumes**.

Impact

An unauthorized user can manipulate the username parameter to retrieve another user's network hierarchy.

The exposed information includes:

- Usernames
- Wallet addresses
- Membership tiers
- User ranks
- Personal volume
- Direct and indirect network relationships
- Multiple downstream users' account information
- Organizational/network structure

The vulnerability is particularly significant because a single request can disclose information belonging to **multiple users within the target user's network**, rather than exposing only the target user's own profile.

An attacker could potentially enumerate usernames and systematically map the platform's network structure, identify relationships between users, associate wallet addresses with usernames, and collect membership and volume information.

Combined with the other identified IDOR vulnerabilities affecting profile and financial endpoints, this could allow an attacker to correlate **identity, wallet, network hierarchy, and financial activity** across multiple accounts.

Recommendation

Implement strict **server-side object-level authorization** for the network tree endpoint.

The application should:

1. Authenticate the requesting user before returning network information.
2. Verify that the requester is authorized to view the requested user's network tree.
3. Do not rely on knowledge of a username as authorization.
4. Restrict network visibility according to the application's intended business rules.
5. Avoid exposing wallet addresses and other sensitive attributes of unrelated users where they are not required.
6. Return `403 Forbidden` or an appropriate authorization response when unauthorized access is attempted.

7. Review all endpoints accepting usernames, wallet addresses, user IDs, or similar identifiers for the same authorization weakness.
8. Implement rate limiting and monitoring to detect automated username enumeration and network-tree harvesting.

Expected Result

A user should only be able to access network-tree information they are explicitly authorized to view. Changing the username in the request to another user's username should **not disclose that user's network hierarchy or the associated information of downstream members** without appropriate authorization.

5. Insecure Direct Object Reference (IDOR) / Broken Object Level Authorization – Unauthorized Leadership & Financial Status Exposure

Severity: Critical
CWE: CWE-639 – Authorization Bypass Through User-Controlled Key
OWASP API Security: API1:2023 – Broken Object Level Authorization
Affected Endpoint: `GET /api/network/{walletAddress}/leadership-status`

Description

The `/api/network/{walletAddress}/leadership-status` API endpoint is vulnerable to **Insecure Direct Object Reference (IDOR)** due to insufficient object-level authorization.

The endpoint accepts a wallet address as a user-controlled path parameter and returns leadership status and financial information associated with the specified wallet.

During testing, the wallet address was modified from the initially requested account to another user's wallet address. The application successfully processed the modified request and returned the target user's leadership information with an HTTP `200 OK` response.

No effective authorization control was observed to verify whether the requesting user was permitted to access the requested account's leadership and financial information.

Steps to Reproduce

Step 1 — Request the leadership status for a valid wallet

```
GET /api/network/0x4E6DcF5BbB225FFF0437A358d638bEAD891C07B8/leadership-status HTTP/2
Host: hntr-backend-dev.onrender.com
```

The application returned leadership and financial information associated with the wallet, including:

- Username
- Rank
- Leadership shares
- Pool balance
- Wallet balance
- Estimated payout
- Lifetime payout
- Last payout
- Payout history

Step 2 — Modify the wallet address

The wallet address was changed to another user's wallet:

```
GET /api/network/0x883eae7dc8ae8ee5be23ab552ed36cc9a80c3a17/leadership-status HTTP/2
Host: hntr-backend-dev.onrender.com
```

Step 3 — Observe the response

The application returned:

```
HTTP/2 200 OK
Content-Type: application/json
```

The response disclosed the target user's leadership status:


```
{
  "walletAddress": "0x883eae7dc8ae8ee5be23ab552ed36cc9a80c3a17",
  "username": "user1",
  "rank": "Scout",
  "shares": 0,
  "hasShares": false,
  "totalShares": 16,
  "eligibleUserCount": 2,
  "poolBalanceUSD": 205.47,
  "estimatedPayoutUSD": 0,
  "lifetimePaidUSD": 0
}
```


The response also exposed wallet balance information, including the wallet address associated with the leadership pool and token balances.

Impact

An unauthorized user can manipulate the wallet address to access another user's **leadership status and financial information**.

The exposed information includes:

- Username
- Wallet address
- Leadership rank
- Number of leadership shares
- Total leadership shares
- Eligible user count
- Leadership pool balance
- Wallet balance
- Token balances
- Token contract addresses
- Estimated payout
- Lifetime payout
- Previous payout information
- Payout transaction hashes
- Payout timestamps
- Leadership payout history

The disclosure of **financial information and wallet-related data belonging to another user** creates a significant confidentiality and privacy risk.

Furthermore, the endpoint reveals information about the platform's leadership pool, eligibility calculations, payout structure, and individual user participation. If wallet addresses can be obtained or enumerated, an attacker could potentially automate requests to collect leadership and financial information across multiple accounts.

Recommendation

Implement strict **server-side object-level authorization** for the affected endpoint.

The application should:

1. Authenticate the requesting user before returning leadership information.
2. Verify that the authenticated user is authorized to access the specified wallet's leadership status.
3. Do not rely on knowledge of a wallet address as proof of authorization.
4. Restrict sensitive financial and payout information to the account owner or explicitly authorized roles.
5. Avoid exposing internal wallet addresses and token balances where they are not required by the client.
6. Return `403 Forbidden` or an appropriate authorization response for unauthorized requests.
7. Review other wallet-based endpoints for the same authorization weakness.
8. Implement rate limiting and monitoring to detect automated wallet enumeration and bulk harvesting.

Expected Result

A user should only be able to retrieve their own leadership and financial information, or information for which they have explicit authorization.

Changing the wallet address to another user's wallet should **not disclose that user's leadership status, wallet balances, payout information, or other financial data**.

6. Insecure Direct Object Reference (IDOR) / Broken Object Level Authorization – Unauthorized Achievement & Financial Information Exposure

Severity: Critical
CWE: CWE-639 – Authorization Bypass Through User-Controlled Key
OWASP API Security: API1:2023 – Broken Object Level Authorization
Affected Endpoint: `GET /api/network/{walletAddress}/achievement-status`

Description

The `/api/network/{walletAddress}/achievement-status` API endpoint is vulnerable to **Insecure Direct Object Reference (IDOR)** due to insufficient object-level authorization.

The endpoint accepts a wallet address as a user-controlled path parameter and returns achievement status, rank bonus, wallet balance, and payout information associated with the specified wallet.

During testing, the wallet address was modified from the originally requested account to another user's wallet address. The application successfully processed the modified request and returned the target user's achievement and financial information with an HTTP `200 OK` response.

No effective authorization control was observed to verify whether the requesting user was authorized to access the requested account's achievement and financial information.

Steps to Reproduce

Step 1 — Request achievement status for a valid wallet

```
GET /api/network/0x883eae7dc8ae8ee5be23ab552ed36cc9a80c3a17/achievement-status HTTP/2
Host: hntr-backend-dev.onrender.com
```

The application returned the achievement and payout information associated with the wallet.

Step 2 — Modify the wallet address

The wallet address was changed to another user's wallet:

```
GET /api/network/0x4E6DcF5BbB225FFF0437A358d638bEAD891C07B8/achievement-status HTTP/2
Host: hntr-backend-dev.onrender.com
```

Step 3 — Observe the response

The application returned:

```
HTTP/2 200 OK
Content-Type: application/json
```

The response disclosed the target user's information, including:

```
{
  "walletAddress": "0x4e6dcf5bbb225fff0437a358d638bead891c07b8",
  "username": "adarsh",
  "rank": "Hunter",
  "lifetimePaidUSD": 925,
```

```
  "pendingUSD": 0,
  "payableNowUSD": 0,
  "waitingOnFundingUSD": 0,
  "hasPending": false,
  "hasPaid": true,
  "poolBalanceUSD": 707.5
}
```


The response also exposed wallet balances and achievement bonus records, including payment amounts, timestamps, token information, and transaction hashes.

Impact

Successful exploitation allows an unauthorized user to retrieve another user's **achievement, wallet, and financial information** by modifying the wallet address.

The exposed information includes:

- Username
- Wallet address
- Current rank
- Lifetime rank bonuses
- Pending bonus amounts
- Payable amounts
- Pool balance
- Wallet balances
- Token balances
- Token contract addresses
- Achievement bonus history
- Bonus amounts
- Bonus status
- Bonus creation and payment timestamps
- Payment transaction hashes

The endpoint also discloses detailed information regarding the user's participation in the platform's rank-based financial reward system.

An attacker who can obtain or enumerate wallet addresses could potentially automate requests and collect achievement and financial information belonging to multiple users. This creates a significant confidentiality and privacy risk, particularly because the exposed information includes **financial records and wallet-related information**.

Recommendation

Implement strict **server-side object-level authorization** for the affected endpoint.

The application should:

1. Authenticate the requesting user before returning achievement information.
2. Verify that the authenticated user is authorized to access the requested wallet's achievement status.
3. Never treat knowledge of a wallet address as sufficient authorization.
4. Restrict financial, wallet-balance, and payout information to the account owner or explicitly authorized roles.
5. Minimize sensitive information returned by the API.
6. Return `403 Forbidden` or an appropriate authorization response for unauthorized requests.

7. Review all wallet-based endpoints for similar IDOR/BOLA vulnerabilities.
8. Implement rate limiting and monitoring to detect wallet enumeration and automated data harvesting.

Expected Result

Changing the wallet address to another user's wallet should **not disclose that user's achievement status, wallet balances, bonus history, payout information, or transaction details** unless the requesting user has explicit authorization to access the information.

7. Insecure Direct Object Reference (IDOR) / Broken Object Level Authorization – Unauthorized Notification & Financial Information Exposure

Severity: Critical
CWE: CWE-639 – Authorization Bypass Through User-Controlled Key
OWASP API Security: API1:2023 – Broken Object Level Authorization
Affected Endpoint: `GET /api/network/{walletAddress}/notifications`

Description

The `/api/network/{walletAddress}/notifications` API endpoint is vulnerable to **Insecure Direct Object Reference (IDOR)** due to insufficient object-level authorization.

The endpoint accepts a wallet address as a user-controlled path parameter and returns notifications associated with the specified wallet.

During testing, the wallet address was modified from the originally requested account to another user's wallet address. The application successfully processed the modified request and returned the target user's notification history with an HTTP `200 OK` response.

The returned information contains sensitive account, financial, transaction, membership, rank, and network-related details. No effective authorization control was observed to verify whether the requesting user was authorized to access the requested wallet's notifications.

Steps to Reproduce

Step 1 — Request notifications for a valid wallet

```
GET /api/network/0x883eae7dc8ae8ee5be23ab552ed36cc9a80c3a17/notifications?limit=50 HTTP/2
Host: hntr-backend-dev.onrender.com
```

The application returned the notification history associated with the wallet.

Step 2 — Modify the wallet address

The wallet address was changed to another user's wallet:

```
GET /api/network/0x4E6DcF5BbB225FFF0437A358d638bEAD891C07B8/notifications?limit=50 HTTP/2
Host: hntr-backend-dev.onrender.com
```

Step 3 — Observe the response

The modified request returned:

```
HTTP/2 200 OK
Content-Type: application/json
```

The response contained notifications belonging to the other user, including membership upgrades, commission earnings, commission claims, leadership payouts, rank changes, achievement payouts, and associated transaction information.

For example, the response exposed details such as:

- Membership upgrade amounts
- Commission amounts and locked amounts
- Source usernames and wallet addresses
- Leadership payout amounts
- Achievement bonus amounts
- Rank changes
- Transaction hashes
- Transaction timestamps
- Membership tiers
- Leadership shares
- Unread notification count

The response also disclosed transaction and financial information belonging to users referenced within the target user's network.

Impact

Successful exploitation allows an unauthorized user to retrieve another user's **private notification history and associated account information** by modifying the wallet address.

The exposed information includes:

- Account activity notifications
- Membership purchases and upgrades
- Membership tiers and upgrade history
- Rank changes and previous ranks
- Leadership share information
- Commission earnings
- Claimable and locked commission amounts
- Commission source usernames
- Commission source wallet addresses
- Achievement bonus payments
- Leadership payout information
- Transaction hashes
- Transaction timestamps
- Payment amounts
- Token information
- Unread notification count

The notification history effectively provides a **timeline of the target user's financial and account activity**.

Because commission notifications also identify other participants through usernames and wallet addresses, exploitation can expose information beyond the directly targeted account.

An attacker capable of obtaining or enumerating wallet addresses could potentially automate requests and harvest notification histories across multiple users, resulting in significant unauthorized disclosure of **financial, account, and network information**.

Recommendation

Implement strict **server-side object-level authorization** for the affected endpoint.

The application should:

1. Authenticate the requesting user before returning notifications.
2. Verify that the authenticated user is authorized to access the requested wallet's notification history.
3. Do not treat possession or knowledge of a wallet address as sufficient authorization.
4. Restrict notification data to the account owner or explicitly authorized roles.
5. Minimize sensitive information included in notifications and API responses.
6. Avoid unnecessarily exposing third-party wallet addresses and usernames within notification metadata.
7. Return `403 Forbidden` or an equivalent authorization response when unauthorized access is attempted.
8. Review all wallet-based endpoints for similar IDOR/BOLA vulnerabilities.
9. Implement rate limiting and monitoring to detect automated wallet enumeration and bulk notification harvesting.

Expected Result

Changing the wallet address to another user's wallet should **not return that user's notifications, financial activity, transaction information, membership history, or network-related information** unless the requesting user has explicit authorization to access the data.

8. Insecure Direct Object Reference (IDOR) / Broken Object Level Authorization – Unauthorized Rewards & Financial Information Exposure

Severity: Critical
CWE: CWE-639 – Authorization Bypass Through User-Controlled Key
OWASP API Security: API1:2023 – Broken Object Level Authorization
Affected Endpoint: `GET /api/network/{walletAddress}/rewards-summary`

Description

The `/api/network/{walletAddress}/rewards-summary` API endpoint is vulnerable to **Insecure Direct Object Reference (IDOR)** due to insufficient object-level authorization.

The endpoint accepts a wallet address as a user-controlled path parameter and returns detailed rewards and financial information associated with the specified wallet.

During testing, the wallet address was modified from the originally requested account to another user's wallet address. The application successfully processed the modified request and returned the target user's rewards summary with an HTTP `200 OK` response.

The response disclosed account, network, rewards, and financial information belonging to the targeted user without demonstrating that the requesting user was authorized to access it.

Steps to Reproduce

Step 1 — Request the rewards summary for a valid wallet

```
GET /api/network/0x4E6DcF5BbB225FFF0437A358d638bEAD891C07B8/rewards-summary HTTP/2
Host: hntr-backend-dev.onrender.com
```

The application returned the rewards summary associated with the wallet, including rank, membership tier, network size, team volume, reward progress, claimable rewards, locked rewards, and token balances.

Step 2 — Modify the wallet address

The wallet address was changed to another user's wallet:

```
GET /api/network/0x883eae7dc8ae8ee5be23ab552ed36cc9a80c3a17/rewards-summary HTTP/2
Host: hntr-backend-dev.onrender.com
```

Step 3 — Observe the response

The application returned:

```
HTTP/2 200 OK
Content-Type: application/json
```

The response disclosed the target user's rewards information, including:

```
{
  "walletAddress": "0x883eae7dc8ae8ee5be23ab552ed36cc9a80c3a17",
  "username": "user1",
  "rank": "Scout",
  "tier": "Bronze",
  "teamVolume": 21650,
```

```
  "networkSize": 13,
  "claimableNow": 262,
  "lockedRemaining": 65.5,
  "totalRewarded": 327.5
}
```


The response also disclosed network-leg information and token-specific reward balances.

Impact

Successful exploitation allows an unauthorized user to retrieve another user's **rewards and financial information** by modifying the wallet address.

The exposed information includes:

- Username
- Wallet address
- Membership tier
- Current rank
- Account creation date
- Team volume
- Network size
- Rank progression information
- Current and next rank thresholds
- Network leg volumes
- Reward caps and progress percentages
- Claimable rewards
- Locked rewards
- Total rewards received
- Token-specific claimable balances
- Token-specific locked balances
- Token contract addresses

This information can provide an attacker with detailed insight into a user's **financial rewards, network performance, membership status, and progression within the platform**.

If wallet addresses can be obtained or enumerated, the vulnerability could potentially be automated to collect rewards and financial information across multiple user accounts.

Recommendation

Implement strict **server-side object-level authorization** for the affected endpoint.

The application should:

1. Authenticate the requesting user before returning rewards information.
2. Verify that the authenticated user is authorized to access the specified wallet's rewards summary.
3. Do not treat knowledge of a wallet address as sufficient authorization.
4. Restrict financial and reward information to the account owner or explicitly authorized roles.
5. Minimize sensitive information returned by the API.
6. Return `403 Forbidden` or an equivalent authorization response when unauthorized access is attempted.
7. Review other wallet-based endpoints for similar IDOR/BOLA vulnerabilities.
8. Implement rate limiting and monitoring to detect wallet enumeration and automated data harvesting.

Expected Result

Changing the wallet address to another user's wallet should **not disclose that user's rewards summary, financial balances, network performance, or account progression information** unless the requesting user has explicit authorization to access the data.

Conclusion

The Vulnerability Assessment and Penetration Testing (VAPT) assessment identified multiple **Critical-severity security vulnerabilities** within the tested API endpoints, primarily related to **Broken Object Level Authorization (BOLA) / Insecure Direct Object Reference (IDOR)**.

The identified vulnerabilities allow unauthorized access to information belonging to other users by manipulating user-controlled identifiers such as wallet addresses and usernames. The exposed information includes **Personally Identifiable Information (PII), financial and reward information, transaction history, wallet-related data, network hierarchy, leadership and achievement information, notifications, and account activity**.

The identified issues present a significant confidentiality risk and could potentially be exploited to systematically enumerate and collect sensitive information across multiple user accounts. The concentration of similar authorization weaknesses across multiple endpoints also indicates a broader issue with object-level authorization controls within the API architecture rather than an isolated endpoint-specific flaw.

It is strongly recommended that the development team prioritize remediation of these Critical findings by implementing robust **server-side authentication and object-level authorization controls**, ensuring that users can access only the resources for which they are explicitly authorized. Following remediation, a comprehensive **retest** should be performed to validate that the vulnerabilities have been successfully addressed and that similar authorization issues do not remain across other API endpoints.

Overall, addressing these findings should be treated as a high priority to reduce the risk of unauthorized data disclosure and strengthen the application's overall security posture.